API

An API For Your Finances

ALL TOPICS
SECURITY
SYNC
ACCOUNTS
BUDGETS
FORECAST
INSIGHTS
INVESTMENTS
RULES
BACKUPS
ACCESS CONTROL
PRICING
IOU (SPLIT BILLS)
LOGIN
API
MOBILE
TAGS
TRANSACTIONS
UPLOAD

Our API provides a flexible interface to personalize and customize your financial information. Read the documentation below and get started with sample code in Ruby, Python, PHP or Perl.

API DOCUMENTATION

API calls follow the format:

```
https://www.buxfer.com/api/[command]?token=login_token[&params]
```

Valid values for **command**:

`login`	get a unique 'token' which must included on all future requests
`transaction_add`	add a transaction
`transaction_edit`	edit a transaction
`transaction_delete`	delete a transaction
`upload_statement`	upload a bank or credit card statement
`transactions`	list of transactions
`accounts`	list of accounts with balances
`loans`	list of your loans
`tags`	list of your transaction tags
`budgets`	list of your budgets
`reminders`	list of your reminders
`groups`	list of your groups
`contacts`	list of your contacts

All commands, except the **`login`** command, must specify a *token* parameter returned by the **`login`** command in the event of a successful login.

The response element contains a **`status`** element which reports the status of the API call. If the call succeeded, it's content is **`OK`**, otherwise it is **`ERROR: [error_description]`**.

`login`

This command only looks at POST parameters and discards GET parameters.

Before accessing any of the API commands, you must obtain a unique, ephemeral token by logging in to the API service.

On successful login, the command returns a *token* which must be included in all future API requests.

Login using email address and password

```
POST /api/login email=john@doe.com&password=dohdoh
```

See sample output

`transaction_add`

This command only looks at POST parameters and discards GET parameters.

The following parameters are accepted:

- **`description`**
- **`amount`**
- **`accountId`**
- **`fromAccountId`**
- **`toAccountId`**
- **`date:`** `YYYY-MM-DD`
- **`tags`**: comma-separated tag names
- **`type:`** `expense | income | refund | transfer | investment_buy | investment_sell | investment_dividend | capital_gain | capital_loss | sharedBill | paidForFriend | settlement | loan`
- **`status:`** `cleared | pending`

Parameters for `type = sharedBill`

- **`payers:`** `[{"email", "amount"}]` JSON-formatted array
- **`sharers:`** `[{"email", "amount"}]` JSON-formatted array
- **`isEvenSplit:`** `true | false`

Parameters for `type = loan`

- **`loanedBy:`** uid | email
- **`borrowedBy:`** uid | email

Parameters for `type = paidForFriend`

- **`paidBy:`** uid | email
- **`paidFor:`** uid | email

Got a pending paycheck for $4000 into bank account

```
POST /api/transaction_add description=Paycheck&type=income&amount=4000&status=pending
```

Loaned $8 to a friend for lunch

```
POST /api/transaction_add
description=Loan&amount=8&type=loan&loanedBy=myself@email.com&borrowedBy=myfrield@email.com
```

Shared grocery bill with roommates

```
POST /api/transaction_add description=Grocery&amount=126&type=sharedBill&sharers=
[{"email":"myself@email.com"},{"email":"roommate.1@email.com"},
{"email":"roommate.2@email.com"}]&payers=[{"email":"myself@email.com","amount":100},
{"email":"roommate.1@email.com","amount":26}]
```

See sample output

`transaction_edit`

This command only looks at POST parameters and discards GET parameters.

The following parameters are accepted:

- **`id`**
- **`description`**
- **`amount`**
- **`accountId`**

- **`fromAccountId`**
- **`toAccountId`**
- **`date:`** `YYYY-MM-DD`
- **`tags`**: comma-separated tag names
- **`type:`** `income | expense | transfer | refund | sharedBill | paidForFriend | loan`
- **`status:`** `cleared | pending`

Parameters for `type = sharedBill`

- **`payers:`** `[{"email", "amount"}]` JSON-formatted array
- **`sharers:`** `[{"email", "amount"}]` JSON-formatted array
- **`isEvenSplit:`** `true | false`

Parameters for `type = loan`

- **`loanedBy:`** uid | email
- **`borrowedBy:`** uid | email

Parameters for `type = paidForFriend`

- **`paidBy:`** uid | email
- **`paidFor:`** uid | email

Got a pending paycheck for $4000 into bank account
```
POST /api/transaction_edit id=18282&description=Paycheck&type=income&amount=4000&status=pending
```

Loaned $8 to a friend for lunch
```
POST /api/transaction_edit
id=3281&description=Loan&amount=8&type=loan&loanedBy=myself@email.com&borrowedBy=myfrield@email.com
```

Shared grocery bill with roommates
```
POST /api/transaction_edit id=28329&description=Grocery&amount=126&type=sharedBill&sharers=
[{"email":"myself@email.com"},{"email":"roommate.1@email.com"},
{"email":"roommate.2@email.com"}]&payers=[{"email":"myself@email.com","amount":100},
{"email":"roommate.1@email.com","amount":26}]
```

See sample output

`transaction_delete`

This command only looks at POST parameters and discards GET parameters.

The following parameters are accepted:

- **`id`**

Delete a transaction by id
```
POST /api/transaction_delete id=18282
```

See sample output

`upload_statement`

This command only looks at POST parameters and discards GET parameters.

Parameters:

- **`accountId`**
- **`statement`**: the text of the statement to be uploaded
- **`dateFormat`**: (optional) date format used in the statement (one of "MM/DD/YYYY" or "DD/MM/YYYY")

See sample output

`transactions`

You can restrict the set of transactions returned by specifying **`parameters`**. The permitted parameters are:

- **`accountId`** `OR` **`accountName`**
- **`tagId`** `OR` **`tagName`**
- **`startDate`** `AND` **`endDate`** `OR` **`month`**: date can be specified as "10 feb 2008", or "2008-02-10". month can be specified as "feb08", "feb 08", or "feb 2008".
- **`budgetId`** `OR` **`budgetName`**
- **`contactId`** `OR` **`contactName`**
- **`groupId`** `OR` **`groupName`**
- **`status`**: pending | reconciled | cleared
- **`page`**: paginate between results

Each call to this method returns at most 100 transactions matching the specified criteria. If there are more that 100 results, you can specify increasing values of the **page** parameter to page through the results.

Your most recent 100 transactions
`GET /api/transactions`

Previous 100 transactions
`GET /api/transactions?page=2`

Transactions for account 'Amex'
`GET /api/transactions?accountName=Amex`

Transactions for account id f32ade4
`GET /api/transactions?accountId=f32ade4`

Transactions for tag 'Grocery' for January 2008
`GET /api/transactions?transactions?tagName=Grocery&month=jan08`

See sample output

`accounts`

List of all your accounts
`GET /api/accounts`

See sample output

`loans`

List of all your loans with friends
`GET /api/loans`

See sample output

`tags`

List of all your tags
`GET /api/tags`

See sample output

budgets

List of all your budgets
`GET /api/budgets`

See sample output

reminders

List of all your reminders
`GET /api/reminders`

See sample output

groups

List of all your groups
`GET /api/groups`

See sample output

contacts

List of all your contacts
`GET /api/contacts`

See sample output

Sample Code

You can get started with some sample code in some of the popular languages listed below. Each sample program logs in and retrieves the login token, downloads the list of your budgets, and prints the name, spending limit and available balance for the budget for the current period.

- Perl : buxfer_api_client.pl (requires LWP and JSON modules)
- Python : buxfer_api_client.py (requires the simplejson module)
- Ruby : buxfer_api_client.rb (requires the json gem)
- PHP : buxfer_api_client.php (uses the CURL and JSON modules, available by default with most PHP installations after 5.2.1)

Financial information is sensitive and so special care must be taken to make sure it is not inadvertently leaked in any way due to the use of Buxfer's API.

Buxfer's responsibility

We take the following steps to make sure your data stays secure and private:

- **Requiring login tokens for API requests**: In order to foil Cross site request forgery attacks (which all HTTP-Basic-auth based APIs are vulnerable to), we require all API requests to be accompanied with unique, ephemeral tokens which are impossible to forge.

- **Never store any sensitive data**: We do not store your banking credentials, real-life names, phone numbers, addresses or anything that will even remotely identify you. The only thing we store is "description, date and amount" for your transactions.
- **128-bit SSL**: We use industry standard security channels so your data never gets exposed over the wire.

API client's responsibility

API clients must manage Buxfer passwords carefully, and must **NOT** store them in plain-text or in easily retrievable ways. Although Buxfer's API is RESTful and no state is maintained on the server, it is always recommended not to inadvertently expose any transaction information to external parties.

Let's Get Started!

Easy online personal finance, money management, expense tracking & budgeting tools.

CREATE ACCOUNT TRY A DEMO LOGIN

FEATURES
Sync
Insights
Budgets
Prévision
Investissements
Retirement Planner
règles
Devises
Contrôle d'accès
Des sauvegardes
API
Upload
IOUs

PRODUCT
Pricing
Help
API

COMPARE
Mint
YNAB
Personal Capital
Quicken
Spendee
Moneydance
Banktivity
PocketSmith
Microsoft Money
You Need A Budget

COMPANY
About
Privacy
Terms
Blog

© 2024 BUXFER INC.